CASE STUDY 1: UNIVERSITY SCHOLARSHIP ALLOCATION

1.Problem Statement

To check whether a student is eligible for a scholarship based on marks, attendance, and annual family income.

2. Objectives

To determine scholarship eligibility using conditional statements.

3. Assumptions

-Minimum aggregate marks required are 75%.

-Minimum attendance required is 75%.

-Annual family income must not exceed Rs. 2,00,000.

4.Algorithm

1.Start.

2.Read marks, attendance, and annual family income.

3.Check all eligibility conditions.

4.If all conditions are satisfied, display "Eligible for scholarship".

5.Otherwise, display "Not eligible for scholarship".

6.Stop.

5. Flowchart

Start → Input details → Check eligibility conditions → Display result → Stop.

6. Pseudocode

START

READ marks, attendance, income

IF marks >= 75 AND attendance >= 75 AND income <= 200000

  PRINT "Eligible for scholarship"

ELSE

  PRINT "Not eligible for scholarship"

END IF

STOP

7.Python Source Code

```
marks = int(input("Enter aggregate marks: "))
attendance = int(input("Enter attendance percentage: "))
income = int(input("Enter annual family income: "))


if marks >= 75 and attendance >= 75 and income <= 200000:
  print("Eligible for scholarship")
else:
  print("Not eligible for scholarship")
```

8. Test  Cases\

Marks: 85, Attendance: 80, Income: 150000

Expected Output: Eligible

Marks: 70, Attendance: 80, Income: 150000

Expected Output: Not eligible

Marks: 90, Attendance: 70, Income: 100000

Expected Output: Not eligible

Marks: 90, Attendance: 90, Income: 250000

Expected Output: Not eligible

9. output

```
C:\python\.venv\Scripts\python.exe C:\python\case
Enter aggregate marks: 90
Enter attendance percentage: 80
Enter annual family income: 80000
Eligible for scholarship
```

10. Conclusion

The program checks scholarship eligibility using marks, attendance, family income, and conditional statements.

CASE STUDY 2 : MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATIONS

1 . Problem Statement

To select a suitable mobile plan based on the customer's budget, daily data usage, and required validity.

2. Objectives

To compare customer requirements with available plans and display a suitable option.

3. Assumptions

-Plan A costs Rs. 199, provides up to 2 GB per day, and has 28-day validity.

-Plan B costs Rs. 299, provides up to 3 GB per day, and has 56-day validity.

-Plan C costs Rs. 399, provides up to 4 GB per day, and has 84-day validity.

These are illustrative values for the program.

4. Algorithm

1. Start.

2. Read budget, daily data usage, and required validity.

3. Check whether Plan A is suitable.

4. If not, check Plan B and then Plan C.

5. Display the suitable plan or a message if no plan matches.

6.Stop.

5. Flowchart

Start → Input budget, data, and validity → Check Plan A → Check Plan B → Check Plan C → Display result → Stop.

6. Pseudocode

START

READ budget, data, days

IF budget >= 199 AND data <= 2 AND days <= 28

PRINT "Plan A"

ELSE IF budget >= 299 AND data <= 3 AND days <= 56

PRINT "Plan B"

ELSE IF budget >= 399 AND data <= 4 AND days <= 84

PRINT "Plan C"

ELSE

PRINT "No suitable plan"

END IF

STOP

7. PYTHON SOURCE CODE

```
budget = int(input("Enter your budget: "))
usage = int(input("Enter required daily data in GB: "))
days = int(input("Enter required validity in days: "))

if budget >= 199 and usage <= 2 and days <= 28:
   print("Plan A is suitable")
elif budget >= 299 and usage <= 3 and days <= 56:
   print("Plan B is suitable")
elif budget >= 399 and usage <= 4 and days <= 84:
   print("Plan C is suitable")
```

```
else:
   print("No suitable plan available")
```

8. Test Cases

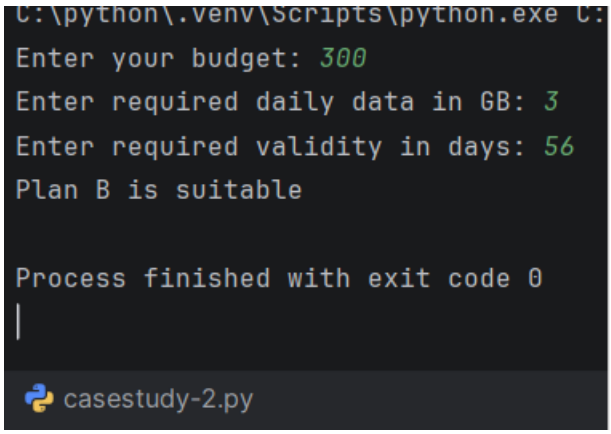
Budget: 200, Data: 2 GB, Validity: 28 days → Plan A

Budget: 300, Data: 3 GB, Validity: 56 days → Plan B

Budget: 400, Data: 4 GB, Validity: 84 days → Plan C

Budget: 150, Data: 2 GB, Validity: 28 days → No suitable plan

9. 0utput




10. Conclusion

The program compares customer requirements with available plans and displays a suitable mobile plan.

CASE STUDY 3: RAILWAY FARE AND RESERVATION CALCULATOR

1. Problem  Statement

To calculate railway fare based on passenger age and travel distance, including a reservation fee.

2. Objectives

To calculate the basic fare, apply age-based concessions, and display the final fare.

3. Assumptions

-Up to 100 km: Rs. 1 per km.

-Above 100 km and up to 300 km: Rs. 1.50 per km.

-Above 300 km: Rs. 2 per km.

-Passengers aged 60 or above receive a 20% concession.

-Children aged 12 or below receive a 50% concession.

-Reservation fee is Rs. 30.

-The selected distance rate applies to the entire distance.

4. Algorithm

1. Start.
2. Read passenger name, age, and distance.
3. Calculate fare according to distance.
4. Apply an age concession if eligible.
5. Add the reservation fee.
6. Display the final fare.
7. STOP

5. Flowchart

Start → Input passenger details → Calculate basic fare → Check age → Apply concession → Add reservation fee → Display final fare → Stop.

6. Pseudocode

```
START
READ name, age, distance

IF distance <= 100
   fare = distance * 1

ELSE IF distance <= 300
   fare = distance * 1.5

ELSE
   fare = distance * 2

END IF

IF age >= 60
   fare = fare * 80 / 100

ELSE IF age <= 12
   fare = fare * 50 / 100
```

END IF

final_fare = fare + 30

PRINT final_fare

STOP

7.Python source code

```
if distance <= 100:
   fare = distance * 1
elif distance <= 300:
   fare = distance * 1.5
else:
   fare = distance * 2

if age >= 60:
   fare = fare * 80 / 100
elif age <= 12:
   fare = fare * 50 / 100

total = fare + 30

print("Passenger:", name)
print("Basic Fare:", fare)
print("Reservation Fee: 30")
print("Total Fare:", total)
```

8. Test Cases

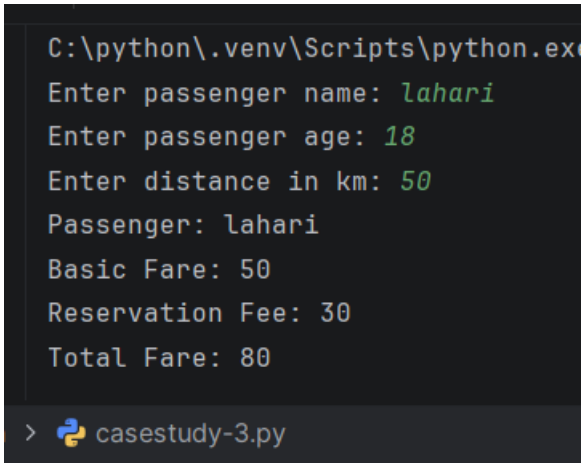
Age: 25, Distance: 80 km → Fare: Rs. 110

Age: 25, Distance: 150 km → Fare: Rs. 255

Age: 65, Distance: 100 km → Fare: Rs. 110

Age: 10, Distance: 100 km → Fare: Rs. 80

9. Output




10. Conclusion

The program calculates railway fare using distance-based rates, age concessions, and a reservation fee.

CASE STUDY 10: CAMPUS EVENT FINANCIAL PLANNING

1. Problem Statement

To calculate the cost and income of a campus event and determine whether the event makes a profit or has a shortfall.

2. Objectives

To calculate event expenses, registration income, sponsorship, and the final balance.

2. Assumptions

-Venue cost is Rs. 10,000.

-Equipment cost is Rs. 5,000.

-Transport cost is Rs. 3,000.

-Catering costs Rs. 150 per participant.

-Certificates cost Rs. 20 per participant.

-Registration income depends on the number of participants and the fee per participant.

3. Algorithm

1. Start.

2. Read the number of participants, registration fee, and sponsorship.

3. Calculate catering and certificate costs.

4. Calculate the total event cost.

5. Calculate registration income and total income.

6. Calculate the balance.

7. Display whether the event is profitable or has a shortfall.

8. Stop.

5. Flowchart

Start → Input participants, fee, and sponsorship → Calculate event costs → Calculate total income → Calculate balance → Check balance → Display result → Stop.

6. Pseudocode

START

READ participants, fee, sponsorship

venue = 10000

equipment = 5000

```
transport = 3000
catering = participants * 150
certificates = participants * 20

total_cost = venue + equipment + transport + catering + certificates

registration = participants * fee
income = registration + sponsorship

balance = income - total_cost

PRINT total_cost, income, balance

IF balance >= 0
   PRINT "Event is profitable"
ELSE
   PRINT "Event has a shortfall"
END IF

STOP
```

7.Python source code

```
catering = participants * 150
certificates = participants * 20

total_cost = venue + equipment + transport + catering + certificates
registration = participants * fee
total_income = registration + sponsorship

balance = total_income - total_cost
```

```
print("Total Cost:", total_cost)
print("Registration Income:", registration)
print("Sponsorship:", sponsorship)
print("Total Income:", total_income)
print("Final Balance:", balance)

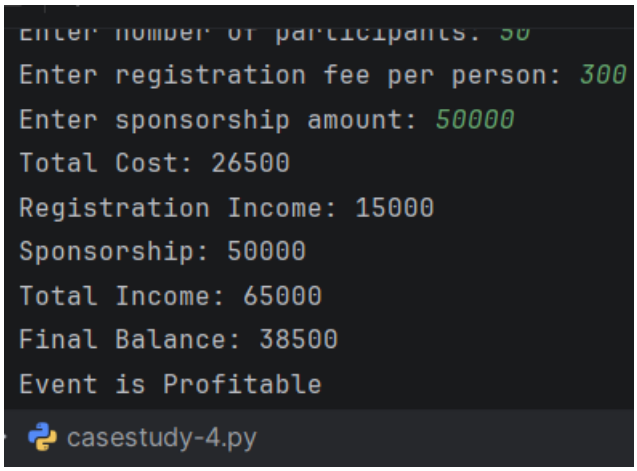
if balance >= 0:
   print("Event is Profitable")
else:
   print("Event has a Shortfall")
```

8. Test Cases

-Participants: 100, Fee: 300, Sponsorship: 10000 → Balance: Rs. 5000 → Profitable

-Participants: 100, Fee: 200, Sponsorship: 5000 → Balance: Rs. -10000 → Shortfall

9. Output




10. Conclusion

The program calculates event expenses and income and determines whether the event is profitable.

CASE STUDY 5: HOSPITAL BILLING AND PATIENT CLASSIFICATION

1. Problem Statement

To calculate a patient's hospital bill and classify it based on the final bill amount.

2. Objectives

To calculate hospital charges, apply a discount, and classify the patient's bill.

3. Assumptions

-The bill includes consultation, diagnosis, and treatment charges.

-A 10% discount is provided when the total bill is Rs. 4,000 or more.

-A final bill of Rs. 3,000 or more is classified as a high bill.

4. Algorithm

1. Start.
2. Read the patient's name and hospital charges.
3. Calculate the total bill.
4. Check whether the total is at least Rs. 4,000.
5. Apply a 10% discount if eligible.
6. Calculate the final bill.
7. Classify the bill as high or normal.
8. Stop.

5. Flowchart

Start → Input patient details → Calculate total → Check discount → Calculate final bill → Classify bill → Display result → Stop.

6 . vPseudocode

```
START

READ name, consultation, diagnosis, treatment

total = consultation + diagnosis + treatment

IF total >= 4000
  discount = total * 10 / 100
ELSE
  discount = 0
END IF

final_bill = total - discount

PRINT total, discount, final_bill

IF final_bill >= 3000
  PRINT "High Bill"
ELSE
  PRINT "Normal Bill"
END IF

STOP
```

7 . Python Source  Code

```
total = consultation + diagnosis + treatment

if total >= 4000:
   discount = total * 10 / 100
else:
   discount = 0

final_bill = total - discount

print("Patient Name:", name)
print("Total Bill:", total)
print("Discount:", discount)
print("Final Bill:", final_bill)

if final_bill >= 3000:
   print("High Bill")
else:
   print("Normal Bill")
```

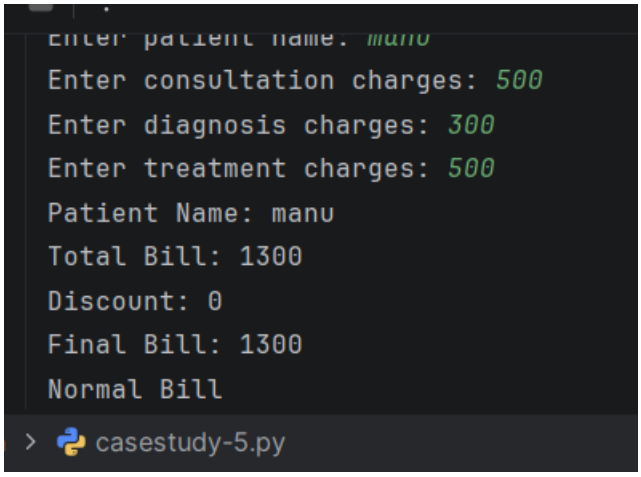
8 .Test  Cases

-Consultation: 500, Diagnosis: 1500, Treatment: 2500 → High Bill

-Consultation: 500, Diagnosis: 500, Treatment: 1000 → Normal Bill

-Consultation: 1000, Diagnosis: 1500, Treatment: 2000 → High Bill

9. Output




10. Conclusion

The program calculates hospital charges, applies a discount, and classifies the final bill using conditional statements.

CASE STUDY 6: EXAMINATION RESULT AND GRACE-MARK VALIDATION

1 . Problem Statement

Determine a Students Result Based on examination marks

2 . Objectives

To classify examination results using conditions.

3 Assumption

-Marks of 40 or above indicate a pass.

-Marks from 35 to 39 indicate that grace marks may be required, subject to college rules.

4. Algorithm

1 .Start.

2 .Read marks.

3 .Check the marks range.

4 .Display the result.

5 .Stop.

5 .Flowchart

Start → Input marks → Check range → Display result → Stop

6 .Pseudo code

```
READ marks

IF marks >= 40
   PRINT "Pass"

ELSE IF marks >= 35
   PRINT "Grace marks may be required"

ELSE
   PRINT "Fail"
```

END IF

2 . Python source code

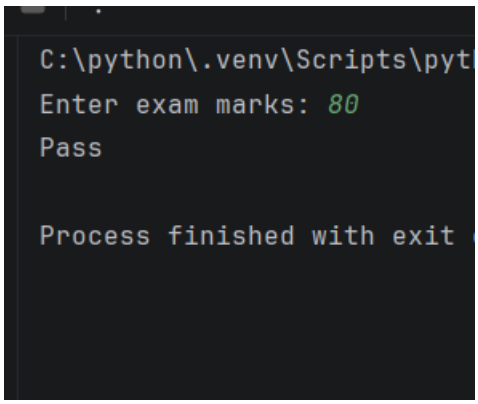
```
marks = int(input("Enter exam marks: "))

if marks >= 40:
   print("Pass")
elif marks >= 35:
   print("Grace marks required to pass")
else:
   print("Fail”)
```

8 . Test cases

| Marks | output |
|---|---|
| 75 | pass |
| 80 | pass |
| 25 | fail |

9. output




10 .conclusion

The program classifies results using conditional statements.

Case Study 7: Attendance Recovery Computation

1. Problem Statement

Calculate attendance percentage and determine how many additional classes are needed to reach 75% attendance.

2. Objectives

To calculate attendance and plan attendance recovery.

3. Assumptions

The minimum required attendance is 75%, and every additional class is attended.

4. Algorithm

1. Start.
2. Read total classes and attended classes.
3. Calculate attendance percentage.
4. If below 75%, calculate the required additional classes.
5. Display the result.
6. Stop.

5. Flowchart

Start → Input classes → Calculate percentage → Check 75% → Display result → Stop.

6 Pseudocode

attendance = attended * 100 / total

PRINT attendance

IF attendance < 75

extra = (0.75 * total - attended) / 0.25

PRINT extra classes required

ELSE

PRINT "Attendance sufficient"

END IF

7.Python Source Code

```
total_classes = int(input("Enter total classes: "))
attended = int(input("Enter classes attended: "))

attendance = attended * 100 / total_classes

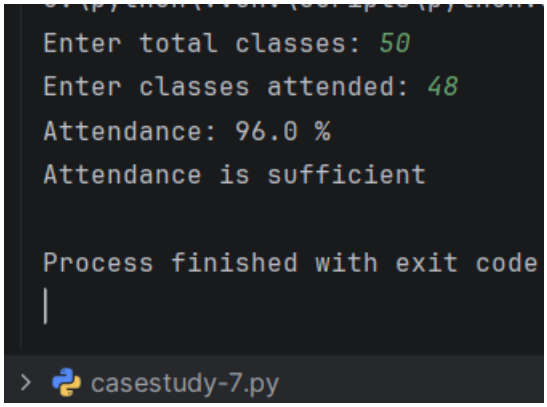
print("Attendance:", attendance, "%")

if attendance >= 75:
   print("Attendance is sufficient")
else:
   needed = (0.75 * total_classes - attended) / 0.25
   print("Extra classes to attend approximately:", int(needed) + 1)
```

8. Test Cases

| Total | Attended | Output |
|---|---|---|
| 100 | 80 | Sufficient |
| 100 | 60 | 60 extra classes |

9. Output




10 .conclusion

The program calculates attendance and estimates the additional classes needed.

Case Study 8: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate ATM operations such as checking balance, depositing money, and withdrawing money.

2. Objectives

To use loops and conditions for basic banking transactions.

3. Assumptions

The initial balance is ₹5000. Withdrawals cannot exceed the available balance.

4. Algorithm

1. Start with an initial balance.

2. Display the ATM menu.

3. Read the user's choice.

4. Perform the selected transaction.

5. Repeat until exit.

6. Stop.

5. Flowchart

Start → Display menu → Read choice → Perform transaction → Repeat or Exit.

6 .Pseudocode

```
Balance=500
REPEAT
   DISPLAY   menu
READ choice
 IF choice = 1
 PRINT balance
 ELSE IF choice = 2
 READ amount
 ADD amount to balance
 ELSE IF choice = 3
 READ amount
 IF amount <= balance
 SUBTRACT amount
 ELSE
 PRINT "Insufficient balance"
 END IF
 ELSE IF choice = 4
 EXIT
 END IF
```

UNTIL exit

7.Python Source Code

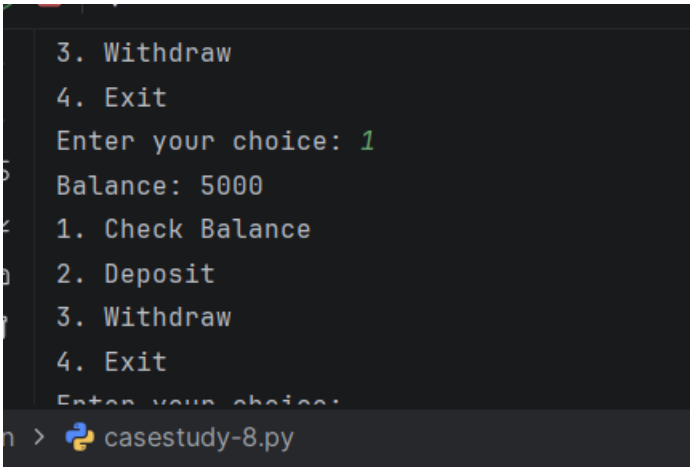
```
choice = int(input("Enter your choice: "))

if choice == 1:
  print("Balance:", balance)
elif choice == 2:
  amount = int(input("Enter deposit amount: "))
  balance += amount
  print("Amount deposited")
elif choice == 3:
  amount = int(input("Enter withdrawal amount: "))
  if amount <= balance:
    balance -= amount
    print("Withdrawal successful")
  else:
    print("Insufficient balance")
elif choice == 4:
  break
else:
  print("Invalid choice")
```

8. Test Cases

| Operation | Amount | Expected Result |
|---|---|---|
| Check balance — | | ₹5000 |
| Deposit | ₹1000 | ₹6000 |
| Withdraw | ₹2000 | ₹4000 |

9 . Output




10 .conclusion

The program simulates basic ATM operations using loops and conditions.

Case Study 9: Warehouse Inventory Reconciliation

1. Problem Statement
Display item stock and update inventory after a sale.

2. Objectives
To manage inventory using lists and list operations.

3. Assumptions
The warehouse stores three items: Pen, Book, and Bag.

. Algorithm
1. Start.
2. Create item and stock lists.
3. Display the stock.
4. Read the sold item and quantity.
5. Check stock and update it.
6. Stop.

5. Flowchart
Start → Create lists → Display stock → Read sale → Check availability → Update stock →
Stop.

6 Pseudocode

```
items = ["Pen", "Book", "Bag"]
stock = [50, 30, 20]
DISPLAY items and stock
READ item, quantity
IF item exists
 FIND item index
 IF quantity <= stock[index]REDUCE stock[index]
 ELSE
 PRINT "Insufficient stock"
 END IF
ELSE
 PRINT "Item not found"
END IF
```

7.Python Source Code

```
items = ["Pen", "Book", "Bag"]
stock = [50, 30, 20]

for i in range(len(items)):
    print(items[i], "Stock:", stock[i])

item = input("Enter item name: ")
quantity = int(input("Enter quantity sold: "))

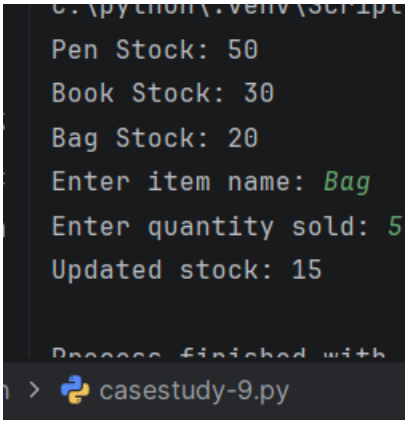
if item in items:
    index = items.index(item)

    if quantity <= stock[index]:
        stock[index] -= quantity
        print("Updated stock:", stock[index])
    else:
        print("Insufficient stock")
else:
    print("Item not found")
```

8. Test Cases

| Item | Initial Stock | Sold | Remaining |
|---|---|---|---|
| Pen | 50 | 10 | 40 |
| Book | 30 | 5 | 25 |

9 Output




10 Conclusion

The program updates warehouse stock using lists and conditions.

Case Study 10: Advanced Numeric Pattern Generator

1. Problem Statement
Generate a number pattern based on the number of rows entered by the user.

2. Objectives
To generate patterns using nested loops.

3. Assumptions
The user enters a positive number of rows.

4. Algorithm

1. Start.
2. Read the number of rows.
3. Use an outer loop for rows.
4. Use an inner loop to print numbers.
5. Stop.

5. Flowchart

Start → Input rows → Outer loop → Inner loop → Print pattern → Stop.

6. Pseudocode

```
READ n
FOR i from 1 to n
 FOR j from 1 to i
PRINT j
 END FOR
 PRINT new line
END FOR
```

7.Python Source Code

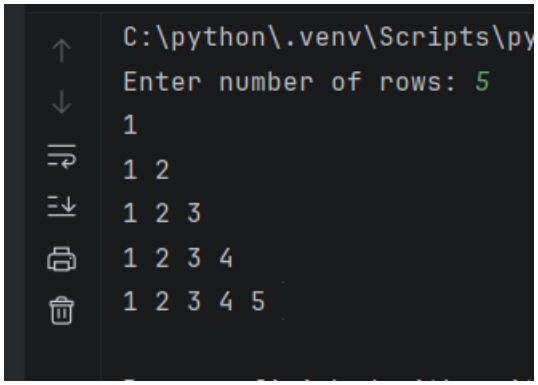
```
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
   for j in range(1, i + 1):
      print(j, end=" ")
   print()
```

8. Test Cases

| Input | Expected Pattern |
|---|---|
| 3 | 1 / 1 2 / 1 2 3 |
| 4 | 1 / 1 2 / 1 2 3 / 1 2 3 4 |

9 Output




10. Conclusion

The program generates numeric patterns using nested loops.